

ECS7001P – NN & NLP

Assignment 2: Multimodal Question Answering – Building a Custom LLaVA Architecture

Carlota Cortés Mir; Student ID: 250711969

Task 1: Image-MCQ Reconstruction

Task 1 focused on reconstructing the missing image-to-caption/MCQ alignment for the validation and test splits. The caption files (*caption_val_student.jsonl* and *caption_test_student.jsonl*) contain *caption_id* values and descriptive captions, while the corresponding images are provided separately in bulk.

To recover the mapping, I implemented *recover_image_id()* using LongCLIP as a shared text-image embedding model. All captions were encoded using the text encoder, while all candidate images were encoded using the vision encoder. The resulting embeddings were projected into a shared embedding space and L2-normalized. After normalization, cosine similarity was computed between every caption-image pair using the dot product, producing a full similarity matrix.

To obtain a globally optimal one-to-one mapping between captions and images, I applied the Hungarian algorithm (*linear_sum_assignment*) to maximize total similarity. This produced a caption to image path mapping, which I used to fill the image field in both the *vqa_val_student.jsonl* and *vqa_test_student.jsonl*, writing the required outputs *vqa_val_student_with_image_ids.jsonl* and *vqa_test_student_with_image_ids.jsonl*.

Finally, the reconstructed validation mapping was evaluated using the provided *evaluate_image_caption_alignment()* function, achieving an accuracy above the required >99% threshold.

Task 2: LLaVA Training & Evaluation

Task 2 focused on building and training a custom LLaVA-style multimodal question-answering model using a CLIP/LongCLIP vision encoder together with the Qwen3-0.6B language model. The model was designed to answer multiple-choice questions by using both the image and the text prompt simultaneously.

To connect the vision encoder and language model, I implemented the *LlavaMultiModalProjector* class as a small MLP (Linear → GELU → Linear). This projection layer converts the image features produced by CLIP into the same embedding space as Qwen, enabling both modalities to work together.

Inside the *CustomLLaVAFForMCQ* model, each image is first processed by the vision encoder to produce a single global image representation. This representation is passed through the projector and added as a visual prefix token before the text embeddings from the MCQ prompt. The combined image and text

embeddings are then passed into the language model so it can use both sources of information when predicting an answer.

For training, the prompts were combined with the correct answer letter (A, B, C, D), and the model was trained using supervised causal language modelling. Label masking (-100) was applied so that loss was only calculated on the answer token and not on the prompt itself. To improve training stability, the image embeddings were also converted to the same datatype as the language model embeddings.

The training setup used projector-only fine-tuning, meaning the vision encoder and language model were frozen while only the projection layer was trained. AdamW optimization, cosine learning-rate scheduling with warmup, gradient accumulation, and gradient clipping were also used during training.

During inference, the model processed the image and MCQ prompt together and predicted the most likely answer choice by comparing the logits for A, B, C, and D. The final predictions were then written to the required output files *vqa_val_student_final.jsonl* and *vqa_test_student_final.jsonl*.

Task 3: Performance Optimization

Task 3 focused on improving the model's validation accuracy through a series of controlled experiments. I selected three categories: architecture tweaks, prompt engineering, and hyperparameter tuning. To ensure fair comparison between experiments, I kept a consistent baseline setup unless specifically testing a hyperparameter change. The baseline configuration used 3 epochs, batch sizes of 2, learning rate = $1e-4$, weight decay = 0.01, gradient accumulation of 4, and 3000 training samples. The baseline model used the original MLP projector together with the standard prompt format and achieved a validation accuracy of 34.00%.

The first improvement explored was prompt engineering. I modified the prompts to include more detailed and directive instructions to encourage the model to reason more carefully about the image and question before selecting an answer. This increased validation accuracy from 34.00% to 34.94%, showing a small but consistent improvement from better prompt design.

Next, I explored architecture improvements by replacing the baseline projector with a gated projector that also included Layer Normalization. The goal was to improve how visual information was mapped into the language-model embedding space. This experiment achieved 35.42% validation accuracy, which was higher than the prompt-only improvement and became the best-performing configuration at that stage.

Then, I combined both approaches by using the detailed prompts together with the gated LayerNorm projector. The combined system achieved 35.03% validation accuracy. Although this was still better than the original baseline, it was slightly

lower than using the gated projector alone, suggesting that the gains from prompt engineering and architecture improvements were not fully additive.

After this, I experimented with hyperparameter tuning. First, I tested a lower learning rate ($5e-5$) together with much larger batch sizes and gradient accumulation. This caused performance to drop significantly to 27.18%, suggesting that optimization became unstable or too slow in the projector-only training setup. Using only the lower learning rate without changing batch size also underperformed at 32.94% accuracy. Returning to the original learning rate of $1e-4$ restored the stronger performance of 35.03%.

I also tested removing weight decay (`weight_decay = 0.0`), which reduced validation accuracy to 33.27%. This suggested that regularization helped the model generalize better and prevented overfitting.

Finally, I investigated the effect of longer training. Using the gated projector with the baseline prompt, increasing training from 3 epochs to 10 epochs improved validation accuracy to 36.70%, which became the best overall result. However, increasing training further to 15 epochs reduced performance slightly to 35.91%, indicating diminishing returns and mild overfitting after extended training.

Overall, the most effective optimization strategy was improving the projector architecture and increasing training duration moderately, rather than aggressively changing optimization settings. The final best-performing configuration achieved 36.70% validation accuracy and was used to generate the final test submission file *vqa_test_student_final.jsonl*.

Although the final validation accuracy of 36.70% is low, the experiments still demonstrate meaningful improvements over the baseline system. The baseline configuration achieved 34.00%, and the optimized model improved this by approximately 2.7 percentage points through architectural changes and training adjustments.

The relatively low overall accuracy is likely due to several constraints in the implementation. First, only the projection layer was trainable, while both the CLIP vision encoder and Qwen language model remained frozen. This limited the model's ability to fully adapt visual and textual representations to the VQA task. Additionally, training was performed on a relatively small subset of the dataset (3000 samples), which restricted the model's ability to generalize.

Experiment Summary:

Experiment	Config	Val Accuracy
Baseline	MLP projector + baseline prompt	34.00%
Experiment 1	MLP projector + detailed prompt	34.94%
Experiment 2	Gated + LayerNorm + baseline prompt	35.42%
Experiment 3	Gated + LayerNorm + detailed prompt	35.03%

Experiment 4	Gated + LayerNorm + detailed prompt + lr=5e-5 + larger batch	27.18%
Experiment 5	Gated + LayerNorm + detailed prompt + lr=5e-5	32.94%
Experiment 6	Gated + LayerNorm + detailed prompt + lr=1e-4	35.03%
Experiment 7	Gated + LayerNorm + detailed prompt + weight_decay=0.0	33.27%
Experiment 8	Gated + LayerNorm + baseline prompt + 10 epochs	36.70%
Experiment 9	Gated + LayerNorm + baseline prompt + 15 epochs	35.91%

Generative AI Disclosure: <https://chatgpt.com/share/6a0623c5-24c8-83eb-b463-2ec433741d71>